



UNIVERSIDAD
CATÓLICA DE
TEMUCO

FACULTAD DE
INGENIERÍA

Universidad Católica de Temuco
Facultad de Ingeniería
Álgebra Lineal para la Computación

Buscador Semántico Simple

Proyecto de Investigación 2 — Representación Vectorial de Texto

Integrantes:

Denys Rodríguez
Paulo Villalobos
Joaquín Valenzuela
David Villegas

Profesora:

Constanza Estrada

23 de junio de 2026

Índice

1. Introducción	2
2. Representación de palabras y documentos como vectores	2
2.1. El Espacio Vectorial Textual	2
2.1.1. El vocabulario y la dimensión del espacio	2
2.1.2. Representación numérica de palabras y documentos	3
2.1.3. Estructura y restricciones algebraicas	3
2.1.4. Interpretación geométrica	4
2.1.5. Dimensión del espacio y dispersión (Sparsity)	4
2.2. La Similitud Coseno	5
2.2.1. Coseno frente a la distancia euclidiana	5
2.2.2. Interpretación geométrica del ángulo y rango de valores	5
2.3. Ejemplo numérico desarrollado	6
2.3.1. Preprocesamiento y construcción del vocabulario	6
2.3.2. Construcción de los vectores TF	6
2.3.3. Cálculo del producto punto a mano	7
2.3.4. Cálculo de las normas a mano	7
2.3.5. Resultado del coseno y su interpretación	7
3. La Matriz Documento-Término	9
3.1. Construcción de la matriz y dispersión	9
3.2. Análisis del corpus del equipo y orquestación	9
4. Implementación y análisis del buscador	11
4.1. El álgebra lineal como motor de búsqueda	11
4.2. Arquitectura del programa	11
4.3. Implementación de los módulos	12
4.3.1. Preprocesamiento del texto	12
4.3.2. Construcción del vocabulario y de la matriz	12
4.3.3. Operaciones vectoriales y similitud coseno	13
4.3.4. Búsqueda: uniendo las piezas	14
4.4. Ejecución del buscador desde la línea de comandos	15
4.5. Análisis de consultas	15
4.6. Efecto del tamaño del vocabulario	16
4.7. Ventajas y limitaciones del coseno	17
4.8. Visualización de los resultados	17
5. Reflexión y conclusiones	19
6. Anexos	21
6.1. Lenguaje y bibliotecas	21
6.2. Código	21
6.3. Bibliografía	21

1. Introducción

Cuando usamos un buscador web o le preguntamos algo a un asistente virtual, esperamos que entienda lo que queremos decir y no solo que busque las palabras exactas que escribimos. Detrás de esa idea hay un problema que parece simple pero no lo es: una computadora no entiende el lenguaje, solo sabe trabajar con números. Por eso, antes de poder comparar dos textos, primero hay que encontrar una forma de convertir las palabras en algo que la máquina sí pueda manipular.

El álgebra lineal entrega una respuesta natural a este problema. Si representamos cada documento como un vector dentro de un espacio de muchas dimensiones, comparar textos deja de ser una tarea de lenguaje y pasa a ser una tarea geométrica: dos textos que hablan de lo mismo apuntan hacia direcciones parecidas, y eso se puede medir con operaciones que ya conocemos, como el producto punto y el ángulo entre vectores. Esta misma idea es la base de los motores de búsqueda, los sistemas de recomendación y buena parte de las aplicaciones de inteligencia artificial que usamos a diario.

A partir de esa idea construimos un buscador semántico simple: un programa que toma un conjunto de textos, los transforma en vectores y, dada una consulta, devuelve los documentos más parecidos usando la similitud coseno. Lo que sigue explica primero las matemáticas que lo hacen posible y después cómo se ven esas ideas funcionando sobre un corpus real de documentos.

2. Representación de palabras y documentos como vectores

2.1. El Espacio Vectorial Textual

Si queremos que una computadora compare diferentes textos y decida si guardan alguna relación temática, nos enfrentamos al problema de que no es posible procesar las palabras directamente de forma cualitativa. De esta forma, el álgebra lineal nos ofrece una solución al permitirnos transformar cada palabra y cada documento en un vector. Como resultado, los textos pasan a ser puntos dentro de un espacio geométrico de muchas dimensiones, lo que permite obtener una representación matemática útil para medir su similitud.

2.1.1. El vocabulario y la dimensión del espacio

El proceso inicia tomando todos los textos de nuestro corpus (el conjunto de documentos) para someterlos a una limpieza o preprocesamiento que elimina puntuación, pasa todo a minúsculas, remueve acentos y quita palabras vacías como “el”, “un” o “de”. La lista que reúne todas las palabras únicas que sobreviven a esta limpieza termina representando nuestro **vocabulario**, al cual denotamos matemáticamente como:

$$V = \{w_1, w_2, \dots, w_n\}$$

donde n terminaría siendo la cantidad total de palabras distintas que pudimos rescatar.

La dimensión del espacio vectorial en el que vamos a trabajar se define a partir del tamaño de este vocabulario, por lo que diríamos que $\dim(V) = n$. Detrás de esto se encuentra la idea de que cada palabra única del vocabulario puede interpretarse como

una dimensión o eje coordenado independiente. Como resultado, si nuestro vocabulario final contara con 5000 términos únicos, los documentos pasarían a representarse como vectores en el espacio real de 5000 dimensiones, $\mathbb{R}^{5000}$.

2.1.2. Representación numérica de palabras y documentos

Para construir los vectores de cada documento, conviene entender primero cómo podemos estructurar algebraicamente las palabras individuales:

1. **One-Hot Encoding (Codificación con un único uno):** Es la forma más simple para vectorizar palabras individuales. A cada palabra $w_i \in V$ le asignamos un vector que tiene un valor de 1 en la posición i y ceros en todas las demás posiciones. De esta forma, cada palabra pasa a estar representada por un vector de la base canónica $\mathbf{e}_i$:

$$\mathbf{e}_i = [0, \dots, 0, \underbrace{1}_{pos. i}, 0, \dots, 0]^T \in \mathbb{R}^n$$

Al usar la base canónica, estamos asumiendo algebraicamente que todas las palabras son ortogonales entre sí. Esto implica que, en este punto, el modelo considera que las palabras son independientes entre sí, por lo que terminaría interpretando que “fútbol” y “balompié” son tan lejanas entre sí como lo serían “fútbol” y “átomo”.

2. **Bag-of-Words (Bolsa de Palabras):** Consiste en crear el vector de un documento sumando los vectores de cada palabra que contiene. Aunque al hacer esto perdemos el orden de las palabras y la gramática, logramos mantener la información de qué palabras aparecen y cuántas veces lo hacen.
3. **Frecuencia de Términos (TF - Term Frequency):** Para esta representación, cada documento d_j pasa a ser un vector $\mathbf{d}_j \in \mathbb{R}^n$ donde cada posición muestra cuántas veces aparece cada palabra del vocabulario en el texto. Esto se escribe matemáticamente como:

$$\mathbf{d}_j = [f_{1j}, f_{2j}, \dots, f_{nj}]^T$$

donde f_{ij} terminaría siendo la cantidad de veces que la palabra w_i aparece en el documento d_j .

Significado práctico y normalización: Si usáramos los conteos directos de las palabras, los documentos más largos tendrían números mucho más altos simplemente por ser más largos. Esto produciría vectores con longitudes (normas) muy grandes, lo que afectaría las comparaciones. Por lo tanto, dividimos cada frecuencia entre el total de palabras que tiene el documento:

$$\text{TF}(w_i, d_j) = \frac{f_{ij}}{\sum_{k=1}^n f_{kj}}$$

De esta forma, cada coordenada pasa a mostrar qué tan importante es esa palabra en el texto en relación con las demás, lo que permite compararlos de manera justa.

2.1.3. Estructura y restricciones algebraicas

Aunque modelamos los vectores de los documentos dentro del espacio vectorial real $\mathbb{R}^n$, en la práctica nos topamos con un límite muy claro: no es posible que una palabra

aparezca una cantidad negativa de veces en un texto. Por lo tanto, los números de nuestros vectores siempre terminan siendo mayores o iguales a cero ($TF \geq 0$).

Como resultado, los vectores de documentos no ocupan todo el espacio $\mathbb{R}^n$, sino que se quedan en el **ortante no negativo** (que es el equivalente de más dimensiones al primer cuadrante que conocemos en el plano cartesiano $\mathbb{R}^2$), denotado como $\mathbb{R}_+^n$. Aunque tengamos este límite, usamos las reglas de $\mathbb{R}^n$ porque nos permiten realizar operaciones matemáticas muy útiles:

- **Suma vectorial:** Si sumamos dos vectores de documentos, el vector que obtenemos termina representando la combinación de ambos textos, sumando las palabras que aparecen en ellos.
- **Multiplicación por un escalar:** Si multiplicamos el vector de un documento por un número positivo $\alpha > 0$, cambiamos el largo del vector pero mantenemos las proporciones de sus palabras. De esta forma, el vector mantiene la misma dirección temática, cambiando solamente su tamaño.

2.1.4. Interpretación geométrica

En este modelo (conocido como Modelo de Espacio Vectorial o VSM), cada documento puede interpretarse como una flecha (un vector de posición) que sale del origen $(0, 0, \dots, 0)$ y apunta hacia un punto en este espacio de n dimensiones.

De esta forma, la dirección de la flecha pasa a mostrar de qué habla el documento. Si dos textos usan palabras parecidas en proporciones similares, sus vectores van a apuntar en direcciones muy cercanas, formando un ángulo pequeño. En cambio, si no comparten ninguna palabra, sus vectores terminarán siendo perpendiculares u **ortogonales**, lo que significa que forman un ángulo de 90° .

Esta mirada geométrica nos ayuda a entender por qué la distancia en línea recta (la distancia euclidiana) no funciona bien para comparar textos, ya que cambia mucho según qué tan largo sea el documento. En su lugar, preferimos medir qué tan parecidos son comparando el ángulo de los vectores, lo que nos lleva a usar la similitud coseno.

2.1.5. Dimensión del espacio y dispersión (Sparsity)

El uso de un vocabulario grande trae consigo dos efectos geométricos e informáticos que debemos tener en cuenta:

1. **Alta dimensionalidad:** A medida que sumamos textos al proyecto, la dimensión n del espacio vectorial crece rápidamente hasta tener miles de dimensiones. En espacios tan grandes, ocurre lo que se conoce como la “maldición de la dimensionalidad”, donde las distancias en línea recta pierden su utilidad porque casi todos los vectores parecen estar a una distancia muy parecida entre sí. Como resultado, la dirección (el ángulo) pasa a ser una medida mucho más útil para comparar los textos.
2. **Dispersión (Sparsity):** Aunque el espacio vectorial tenga 10000 dimensiones por el tamaño del vocabulario, un documento común y corriente solo va a usar unas 50 o 100 palabras únicas. Por lo tanto, casi todas las coordenadas del vector $\mathbf{d}_j$ terminarán siendo iguales a cero. De esta forma, obtenemos vectores altamente **dispersos** (o *sparse*). Para la programación, esto es una ventaja enorme, ya que

nos permite guardar y procesar únicamente los números distintos de cero, lo que ahorra mucha memoria y tiempo de cálculo.

2.2. La Similitud Coseno

Si ya tenemos los documentos transformados en vectores dentro de nuestro espacio de vocabulario, el siguiente paso es encontrar una forma matemática para medir qué tan parecidos son entre sí. Para esto, la herramienta que nos proporciona la geometría es la similitud coseno. En lugar de comparar los textos por la cantidad total de palabras, esta medida se enfoca en evaluar la dirección de los vectores. La similitud coseno del ángulo θ formado entre dos vectores $\mathbf{u}$ y $\mathbf{v}$ se calcula usando la siguiente relación:

$$\cos(\theta) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|}$$

donde el numerador $\mathbf{u} \cdot \mathbf{v}$ terminaría representando el producto punto (la suma de los productos de cada coordenada), lo que nos indica las palabras que tienen en común. Por otro lado, el denominador $\|\mathbf{u}\| \|\mathbf{v}\|$ muestra la multiplicación del largo o magnitud de ambos vectores.

2.2.1. Coseno frente a la distancia euclidiana

Una pregunta natural que surge en este punto es por qué preferimos calcular el coseno del ángulo en lugar de usar la distancia en línea recta (distancia euclidiana) para comparar los textos. La razón de esta elección tiene que ver con la longitud de los documentos. Si tuviéramos un documento corto y un documento largo que hablan exactamente del mismo tema (por ejemplo, fútbol), el documento largo usaría las palabras con mucha más frecuencia. Como resultado, su vector terminaría siendo mucho más largo, situando su punto muy lejos en el espacio en comparación con el vector del documento corto.

Si usáramos la distancia euclidiana, esta diferencia de largo produciría una distancia muy grande entre ambos puntos, lo que nos llevaría a pensar que son textos muy diferentes, lo que afectaría la precisión de la búsqueda. Al usar la similitud coseno, el cálculo ignora por completo el largo de los vectores y evalúa únicamente la dirección de las flechas. De esta forma, logramos comparar los textos por su temática sin importar si uno es mucho más largo que el otro.

Nota sobre la normalización: Dado que la fórmula de la similitud coseno ya divide el producto punto por el largo de los vectores, realiza su propia normalización geométrica. Esto significa que el resultado final del coseno será exactamente idéntico ya sea que calculemos la fórmula usando conteos crudos de palabras o usando las frecuencias relativas (TF) previamente normalizadas. Sin embargo, decidimos aplicar la normalización TF en un paso previo porque nos permite que los vectores individuales tengan un significado por sí mismos (mostrando la proporción de cada palabra en el texto) y para mantener la coherencia con el procesamiento interno del código del buscador.

2.2.2. Interpretación geométrica del ángulo y rango de valores

El ángulo θ entre los vectores nos da una interpretación geométrica muy directa de la relación entre los documentos, lo que se traduce en un rango de valores específico:

- **Coseno igual a 1:** Ocurre cuando el ángulo es de 0° , lo que implica que ambos vectores apuntan exactamente en la misma dirección. Este valor máximo indica que los documentos comparten las mismas palabras en exactamente las mismas proporciones.
- **Coseno igual a 0:** Ocurre cuando el ángulo es de 90° . En este caso, los vectores terminan siendo perpendiculares u **ortogonales**, lo que significa que los documentos no comparten ni una sola palabra de nuestro vocabulario y, por lo tanto, no tienen ninguna relación temática.
- **Coseno igual a -1:** Ocurre cuando el ángulo es de 180° , lo que indicaría que los vectores apuntan en direcciones opuestas. Sin embargo, dado que en el análisis de textos trabajamos en el ortante no negativo $\mathbb{R}_+^n$, es imposible que una palabra tenga una frecuencia negativa. Como resultado, los vectores nunca pueden apuntar en direcciones opuestas, lo que significa que la similitud coseno en nuestro buscador semántico queda limitada estrictamente al rango entre 0 y 1.

2.3. Ejemplo numérico desarrollado

Para ilustrar de forma práctica cómo funcionan estos conceptos de álgebra lineal en la comparación de textos, vamos a desarrollar un ejemplo completo paso a paso usando dos documentos muy sencillos creados para este fin:

- Documento 1 (d_1): “el gato come pescado”
- Documento 2 (d_2): “el perro come carne”

2.3.1. Preprocesamiento y construcción del vocabulario

Antes de construir los vectores, debemos limpiar los documentos. En este caso, eliminamos la palabra “el” por ser una palabra vacía (stopword) que no aporta significado temático. Como resultado de esta limpieza, los textos quedan reducidos a los siguientes términos:

- Documento 1 (d_1): {gato, come, pescado}
- Documento 2 (d_2): {perro, come, carne}

Al reunir todas las palabras únicas de ambos textos y ordenarlas alfabéticamente, obtenemos el vocabulario V :

$$V = \{\text{carne, come, gato, perro, pescado}\}$$

donde el tamaño de nuestro vocabulario terminaría siendo $n = 5$. Esto implica que trabajaremos en un espacio vectorial de cinco dimensiones, $\mathbb{R}^5$.

2.3.2. Construcción de los vectores TF

Con el vocabulario ordenado, pasamos a representar cada documento como un vector, contando la presencia de cada palabra y normalizando el resultado según el total de términos de cada documento (que en ambos casos es igual a 3):

Para el Documento 1 (d_1), las palabras “come”, “gato” y “pescado” aparecen una vez, mientras que “carne” y “perro” no aparecen. De esta forma, el vector normalizado $\mathbf{d}_1$ pasa a ser:

$$\mathbf{d}_1 = \left[0, \frac{1}{3}, \frac{1}{3}, 0, \frac{1}{3}\right]^T$$

Para el Documento 2 (d_2), las palabras “carne”, “come” y “perro” aparecen una vez, mientras que “gato” y “pescado” no aparecen. Esto nos permite generar el vector normalizado $\mathbf{d}_2$:

$$\mathbf{d}_2 = \left[\frac{1}{3}, \frac{1}{3}, 0, \frac{1}{3}, 0\right]^T$$

2.3.3. Cálculo del producto punto a mano

Para calcular el producto punto entre ambos vectores, multiplicamos sus componentes correspondientes y sumamos los resultados:

$$\begin{aligned}\mathbf{d}_1 \cdot \mathbf{d}_2 &= \left(0 \times \frac{1}{3}\right) + \left(\frac{1}{3} \times \frac{1}{3}\right) + \left(\frac{1}{3} \times 0\right) + \left(0 \times \frac{1}{3}\right) + \left(\frac{1}{3} \times 0\right) \\ \mathbf{d}_1 \cdot \mathbf{d}_2 &= 0 + \frac{1}{9} + 0 + 0 + 0 = \frac{1}{9}\end{aligned}$$

El producto punto da como resultado $\frac{1}{9}$, lo que muestra la coincidencia de la palabra “come” (la única coordenada donde ambos vectores tienen valores distintos de cero).

2.3.4. Cálculo de las normas a mano

Para calcular la norma (o longitud) de cada vector, elevamos al cuadrado cada componente, sumamos estos valores y obtenemos la raíz cuadrada del resultado:

Para el vector $\mathbf{d}_1$:

$$\|\mathbf{d}_1\| = \sqrt{0^2 + \left(\frac{1}{3}\right)^2 + \left(\frac{1}{3}\right)^2 + 0^2 + \left(\frac{1}{3}\right)^2} = \sqrt{\frac{1}{9} + \frac{1}{9} + \frac{1}{9}} = \sqrt{\frac{3}{9}} = \sqrt{\frac{1}{3}} = \frac{1}{\sqrt{3}} \approx 0,577$$

Para el vector $\mathbf{d}_2$, el procedimiento es el mismo al tener la misma cantidad de palabras:

$$\|\mathbf{d}_2\| = \sqrt{\left(\frac{1}{3}\right)^2 + \left(\frac{1}{3}\right)^2 + 0^2 + \left(\frac{1}{3}\right)^2 + 0^2} = \sqrt{\frac{1}{9} + \frac{1}{9} + \frac{1}{9}} = \sqrt{\frac{3}{9}} = \sqrt{\frac{1}{3}} = \frac{1}{\sqrt{3}} \approx 0,577$$

2.3.5. Resultado del coseno y su interpretación

Una vez calculados el producto punto y las normas, usamos la fórmula de la similitud coseno para obtener el valor final:

$$\cos(\theta) = \frac{\mathbf{d}_1 \cdot \mathbf{d}_2}{\|\mathbf{d}_1\| \|\mathbf{d}_2\|} = \frac{\frac{1}{9}}{\left(\frac{1}{\sqrt{3}}\right) \times \left(\frac{1}{\sqrt{3}}\right)} = \frac{\frac{1}{9}}{\frac{1}{3}} = \frac{3}{9} = \frac{1}{3} \approx 0,333$$

El valor obtenido para la similitud coseno termina siendo exactamente $\frac{1}{3} \approx 0,333$. Dado que el rango de la similitud coseno va desde 0 hasta 1 en nuestro espacio de documentos, este resultado muestra una coincidencia baja o moderada entre los textos.

Geométricamente, esto significa que los vectores están separados por un ángulo de aproximadamente $70,5^\circ$ ($\arccos(0,333) \approx 70,5^\circ$). Desde el punto de vista del significado, el resultado es coherente: ambos documentos comparten únicamente la acción (la palabra “come”), pero difieren en el sujeto (“gato” frente a “perro”) y en el objeto (“pescado” frente a “carne”). De esta forma, el modelo matemático es capaz de identificar que los textos guardan cierta relación, pero que no son idénticos.

3. La Matriz Documento-Término

3.1. Construcción de la matriz y dispersión

Una vez que hemos representado cada documento individual como un vector matemático, podemos agrupar todos los textos de nuestro corpus en una sola gran estructura algebraica conocida como la matriz documento-término. Esta matriz se construye alineando los vectores de los documentos uno debajo del otro. Como resultado, obtenemos una matriz bidimensional donde cada fila representa un documento completo y cada columna representa una palabra única del vocabulario.

Si llamamos m a la cantidad de documentos y n a la cantidad de palabras del vocabulario, la matriz resultante tendrá un tamaño de $m \times n$.

Un fenómeno matemático muy particular que ocurre con esta matriz es su alta **dispersión** (o *sparsidad*). Dado que cualquier documento típico utiliza apenas una pequeña fracción de todas las palabras disponibles en el vocabulario general (la mayoría de las palabras de un diccionario no aparecen en un texto corto), casi todos los valores de los vectores serán exactamente cero. Mientras más grande sea el corpus de textos, más palabras nuevas se agregarán al vocabulario general, haciendo que la cantidad de ceros en la matriz crezca enormemente.

3.2. Análisis del corpus del equipo y orquestación

Para poner a prueba nuestro buscador semántico, el equipo de investigación reunió un corpus de 16 documentos cortos. Estos textos se dividen en cuatro temáticas principales: historia, matemáticas, naturaleza y tecnología (cuatro textos por cada tema).

Cada tema aporta cuatro textos breves de un par de oraciones. Por ejemplo, el primer texto de historia describe la civilización egipcia y las pirámides de Giza, mientras que el primer texto de matemáticas habla del álgebra lineal, los espacios vectoriales y las matrices. Los 16 documentos que forman el corpus son los siguientes:

- **Historia:** civilización egipcia, Imperio Romano, Revolución Francesa y Segunda Guerra Mundial.
- **Matemáticas:** álgebra lineal, matrices, producto punto y similitud coseno.
- **Naturaleza:** ecosistemas, cambio climático, bosques tropicales y energías renovables.
- **Tecnología:** inteligencia artificial, motores de búsqueda, procesamiento de lenguaje natural y Python.

Al aplicar el proceso de limpieza y extracción de palabras (preprocesamiento) sobre los 16 archivos de texto, nuestro sistema detectó un vocabulario exacto de 250 palabras únicas. Estas palabras son las que sobreviven a la limpieza y se ordenan alfabéticamente; las primeras del vocabulario son, por ejemplo, “acueductos”, “álgebra”, “algoritmos”, “ángulo”, “aprendizaje” y así sucesivamente hasta completar los 250 términos. Con estos datos, la matriz documento-término generada por el programa terminó teniendo un tamaño de 16×250 : una fila por cada documento y una columna por cada palabra del vocabulario. Al analizar internamente los valores de esta matriz, observamos que el nivel de dispersión alcanzó un 92.80%; es decir, más de nueve de cada diez celdas en la matriz

albergan un cero. La forma completa de esta matriz se puede apreciar de manera visual más adelante, en el mapa de calor de la sección de resultados.

El proceso completo para leer los archivos, limpiarlos y construir la matriz de frecuencias se puede observar en el siguiente fragmento de código de nuestro proyecto:

```
import glob
from src.core.preprocessing import preprocess
from src.core.vectorizer import build_vocabulary, build_tf_matrix

corpus_files = sorted(glob.glob('data/corpus/*.txt'))
documentos = []

for filepath in corpus_files:
    with open(filepath, 'r', encoding='utf-8') as f:
        texto = f.read()
        tokens = preprocess(texto)
        documentos.append(tokens)

vocabulario = build_vocabulary(documentos)
matriz_documento_termino = build_tf_matrix(documentos, vocabulario)
```

4. Implementación y análisis del buscador

4.1. El álgebra lineal como motor de búsqueda

La idea detrás de nuestro buscador es relativamente directa: si los documentos y las consultas están representados como vectores en el mismo espacio, comparar una consulta contra un conjunto de documentos se reduce a calcular ángulos. Mientras más pequeño sea el ángulo entre dos vectores, más similares son los textos que representan.

Esta forma de comparar tiene una ventaja algebraica concreta. En lugar de recorrer los documentos uno a uno y revisar cuántas palabras coinciden manualmente, podemos calcular la similitud coseno entre el vector de la consulta y cada fila de la matriz documento-término. El resultado es un vector de scores, donde cada posición indica qué tan relevante es el documento correspondiente para la consulta recibida. Ordenar ese vector de mayor a menor basta para producir el ranking final.

El hecho de que la similitud coseno dependa únicamente de la dirección de los vectores (y no de su magnitud) es especialmente útil aquí. Un documento largo que repite las palabras de la consulta muchas veces no debería estar automáticamente por encima de un documento corto que las menciona con igual proporción. El coseno normaliza esta diferencia de forma natural.

4.2. Arquitectura del programa

El programa se organiza en módulos con responsabilidades bien delimitadas. Esta separación facilita el desarrollo por partes y permite probar cada componente de forma independiente.

- **preprocessing.py**: Recibe un texto crudo y lo convierte en una lista de tokens limpios. Para esto, normaliza el texto a minúsculas, elimina acentos mediante la descomposición Unicode de caracteres, extrae solo las secuencias alfabéticas y filtra las palabras vacías definidas en la lista de stopwords.
- **vectorizer.py**: A partir de los documentos ya preprocesados, construye el vocabulario ordenado y genera la matriz documento-término. Implementa dos estrategias de vectorización: frecuencia de términos (*TF*) y TF-IDF, ambas implementadas directamente con NumPy.
- **similarity.py**: Implementa las operaciones vectoriales básicas a mano: el producto punto, la norma euclidiana y la similitud coseno. También incluye una función que calcula la similitud de una consulta contra todas las filas de la matriz en una sola llamada.
- **search.py**: Une todo lo anterior. Construye el vocabulario y la matriz a partir del corpus, vectoriza la consulta con el mismo proceso, calcula la similitud coseno contra cada documento y devuelve los más parecidos ordenados por score.

El flujo completo del programa es: cargar los archivos del corpus, preprocesar cada texto, construir el vocabulario y la matriz, recibir una consulta, vectorizarla con el mismo proceso, calcular su similitud contra todos los documentos y devolver los resultados ordenados por score.

4.3. Implementación de los módulos

A continuación se muestra el código real de cada módulo, comentado para explicar cómo se conecta con las ideas matemáticas de las secciones anteriores. Todo el cálculo vectorial está hecho directamente con NumPy, sin usar bibliotecas que ya traigan la vectorización resuelta.

4.3.1. Preprocesamiento del texto

Antes de poder vectorizar, hay que dejar cada texto en una forma uniforme. El módulo `preprocessing.py` se encarga de eso en tres pasos: normaliza (minúsculas y sin acentos), separa en palabras y descarta las stopwords. La normalización de acentos se hace con la descomposición Unicode NFKD, que separa cada letra acentuada en la letra base más un signo aparte; luego se eliminan esos signos (categoría `Mn`).

```
1 import unicodedata
2 import re
3
4 def normalize(text: str) -> str:
5     text = text.lower()
6     te = unicodedata.normalize('NFKD', text) # separa letra de acentos
7     result = ""
8     for letra in te:
9         categoria = unicodedata.category(letra)
10        if categoria != 'Mn': # si no es un acento, se conserva
11            result += letra
12    return result
13
14 def tokenize(text: str) -> list[str]:
15     # \w+ extrae secuencias de letras/numeros; se descartan tokens de 1
16     # caracter
17     tokens = [token for token in re.findall(r'\w+', text) if len(token) >
18               1]
19     return tokens
20
21 def remove_stopwords(tokens, stopwords=STOPWORDS) -> list[str]:
22     return list(filter(lambda token: token not in stopwords, tokens))
23
24 def preprocess(text: str, stopword=STOPWORDS) -> list[str]:
25     return remove_stopwords(tokenize(normalize(text)), stopword)
```

Listing 1: Limpieza y tokenización del texto (`preprocessing.py`).

El resultado de `preprocess` es la lista de palabras con significado de un documento, que es justo lo que necesita el siguiente módulo para contar frecuencias.

4.3.2. Construcción del vocabulario y de la matriz

El módulo `vectorizer.py` toma los documentos ya preprocesados y construye dos cosas: el vocabulario (todas las palabras únicas ordenadas alfabéticamente) y la matriz documento-término. La matriz se llena contando cuántas veces aparece cada palabra del vocabulario en cada documento, que es exactamente la representación TF descrita en la primera sección.

```
1 import numpy as np
```

```

2
3 def build_vocabulary(documents: list[list[str]]) -> list[str]:
4     vocabulary = []
5     documents = sum(documents, []) # une todas las listas en una sola
6     for token in documents:
7         if token not in vocabulary:
8             vocabulary.append(token)
9     return sorted(vocabulary)
10
11 def build_tf_matrix(documents, vocabulary) -> np.ndarray:
12     matrix = np.zeros((len(documents), len(vocabulary)))
13     for i in range(len(documents)):
14         for j in range(len(vocabulary)):
15             matrix[i, j] = documents[i].count(vocabulary[j])
16     return matrix

```

Listing 2: Vocabulario y matriz de frecuencias (vectorizer.py).

Sobre esta base se implementa también TF-IDF. La idea es ponderar cada frecuencia por el factor IDF $\log(N/df)$, donde N es el número de documentos y df es en cuántos documentos aparece el término. Así las palabras que están en casi todos los textos pesan poco y las raras pesan más.

```

1 def build_idf_matrix(documents, vocabulary, smoothing=True) -> np.ndarray:
2     idf = np.zeros(len(vocabulary))
3     tf = build_tf_matrix(documents, vocabulary)
4     for j in range(len(vocabulary)):
5         df = np.count_nonzero(tf[:, j]) # en cuantos documentos aparece
6         if smoothing:
7             idf[j] = np.log(len(documents) / (df + 1.0)) # +1 evita div/0
8         elif df > 0:
9             idf[j] = np.log(len(documents) / df)
10    return idf
11
12 def build_tfidf_matrix(documents, vocabulary, smoothing=True) -> np.ndarray:
13     :
14     tf = build_tf_matrix(documents, vocabulary)
15     totales = tf.sum(axis=1, keepdims=True)
16     totales[totales == 0] = 1.0 # documentos vacios: evita div/0
17     tf_norm = tf / totales # frecuencia relativa por documento
18     idf = build_idf_matrix(documents, vocabulary, smoothing)
19     return tf_norm * idf

```

Listing 3: Ponderación TF-IDF (vectorizer.py).

4.3.3. Operaciones vectoriales y similitud coseno

El módulo `similarity.py` implementa a mano las tres operaciones que aparecen en la fórmula del coseno: el producto punto, la norma euclidiana y, con ellas, la similitud coseno. No se usa ninguna función de alto nivel que entregue el coseno directo; se arma con sumas y raíces para dejar en claro la matemática detrás.

```

1 import numpy as np
2
3 def dot_product(u, v) -> float: # u . v = sum(u_i * v_i)
4     u = np.asarray(u, dtype=float)

```

```

5     v = np.asarray(v, dtype=float)
6     if u.shape != v.shape:
7         raise ValueError("Los vectores deben tener la misma dimension")
8     return float(np.sum(u * v))
9
10    def norm(v) -> float:                # ||v|| = sqrt(sum(v_i^2))
11        v = np.asarray(v, dtype=float)
12        return float(np.sqrt(np.sum(v ** 2)))
13
14    def cosine_similarity(u, v) -> float: # cos(theta) = (u . v) / (||u|| ||v||)
15        norm_u = norm(u)
16        norm_v = norm(v)
17        if norm_u == 0.0 or norm_v == 0.0: # vector nulo -> sin direccion
18            definida
19            return 0.0
20        return dot_product(u, v) / (norm_u * norm_v)

```

Listing 4: Producto punto, norma y similitud coseno (similarity.py).

El detalle de devolver 0,0 cuando algún vector es nulo es importante en la práctica: ocurre cuando una consulta no comparte ninguna palabra con un documento, y evita una división por cero. Esto explica por qué en el análisis muchos documentos aparecen con score exactamente cero.

4.3.4. Búsqueda: uniendo las piezas

Finalmente, `search.py` conecta todo. Recibe el corpus ya tokenizado y la consulta tokenizada, construye el vocabulario y la matriz, vectoriza la consulta usando el mismo vocabulario (paso clave: la consulta debe vivir en el mismo espacio que los documentos), calcula la similitud contra cada fila y devuelve los `top_k` resultados ordenados.

```

1  from src.core.vectorizer import build_vocabulary, build_tf_matrix,
   build_idf_matrix
2  from src.core.similarity import cosine_similarity
3
4  def search(corpus_tokens, query_tokens, method="tf", top_k=5):
5      vocabulary = build_vocabulary(corpus_tokens)
6      matriz = build_tf_matrix(corpus_tokens, vocabulary)
7      # la consulta se vectoriza con EL MISMO vocabulario que el corpus
8      query_vector = build_tf_matrix([query_tokens], vocabulary)[0]
9      if method == "idf":
10         idf_vector = build_idf_matrix(corpus_tokens, vocabulary)
11         matriz = matriz * idf_vector
12         query_vector = query_vector * idf_vector
13     elif method != "tf":
14         raise ValueError("El metodo debe ser 'tf' o 'idf'")
15     scores = []
16     for i in range(len(matriz)):
17         sim = cosine_similarity(query_vector, matriz[i])
18         scores.append((i, sim))
19     # ordena de mayor a menor score y devuelve los top_k
20     return sorted(scores, key=lambda x: x[1], reverse=True)[:top_k]

```

Listing 5: Motor de búsqueda (search.py).

4.4. Ejecución del buscador desde la línea de comandos

Sobre estos módulos se construyó una interfaz de línea de comandos (`main.py`) que carga el corpus, recibe una consulta y muestra el ranking. Por ejemplo, para buscar documentos relacionados con inteligencia artificial usando frecuencia de términos, se ejecuta:

```
python -m src.main --query "inteligencia artificial aprendizaje automatico"
--method tf --top-k 5
```

Listing 6: Llamada al buscador desde la terminal.

Y la salida real del programa es:

Resultados (TF):	
Documento	Score
-----	-----
tecnologia_01.txt	0.5213
tecnologia_04.txt	0.2236
historia_01.txt	0.0000
historia_02.txt	0.0000
historia_03.txt	0.0000

Listing 7: Resultado devuelto por el buscador para la consulta anterior.

El programa identifica correctamente los dos documentos de tecnología como los más relevantes y asigna score cero al resto, que no comparte ninguna palabra con la consulta. Este es el mismo procedimiento que se repite para cada una de las consultas analizadas más adelante.

4.5. Análisis de consultas

Para evaluar el comportamiento del buscador, se realizaron cinco consultas representativas sobre el corpus de 16 documentos. Los resultados se obtuvieron usando la estrategia de vectorización TF, que asigna a cada término su frecuencia relativa dentro del documento.

1. Consulta: “pirámides Egipto historia antigua”

El buscador devolvió primero el texto del Imperio Romano (`historia_02`) con un score de 0,26, seguido del texto de la civilización egipcia (`historia_01`) con 0,14; el resto del corpus quedó en cero. Llama la atención que el texto romano gane al egipcio aunque la consulta menciona “pirámides” y “Egipto”. La razón es reveladora: las palabras “historia” y “antigua” aparecen juntas y literales en el texto romano, mientras que “Egipto” nunca aparece como tal (el texto dice “egipcia” y “egipcios”, que para el modelo son palabras distintas). Este caso muestra con claridad que el buscador empareja formas exactas de palabras, no temas ni significados.

2. Consulta: “vectores matrices álgebra lineal”

Los cuatro documentos de matemáticas ocuparon las primeras posiciones, con `matematicas_01` a la cabeza (0,47), seguido de `matematicas_03` (0,27), `matematicas_02` (0,12) y `matematicas_04` (0,11). Ningún documento de otra categoría obtuvo un valor distinto de cero. Este es el comportamiento ideal: la consulta usa el vocabulario propio del tema y el modelo separa limpiamente esa categoría del resto.

3. Consulta: “ecosistema biodiversidad naturaleza”

Solo un documento obtuvo un score distinto de cero: `naturaleza_01` con 0,25. Los otros tres textos de naturaleza quedaron en cero, lo que al principio sorprende. La explicación vuelve a estar en las formas de las palabras: el texto dice “ecosistemas” (plural) y “naturales”, pero la consulta usa “ecosistema” y “naturaleza”, que el modelo trata como términos diferentes. Al final, lo único que coincidió de verdad fue “biodiversidad”. Esto deja en evidencia que el buscador es sensible incluso a detalles como el singular y el plural, porque no aplica ninguna reducción de las palabras a su raíz.

4. Consulta: “inteligencia artificial aprendizaje automático”

Los documentos de tecnología dominaron el ranking, con `tecnologia_01` en primer lugar (0,52) y `tecnologia_04` en segundo (0,22); el resto quedó en cero. Probamos esta misma consulta con TF y con TF-IDF y el ranking fue idéntico, lo que tiene sentido en un corpus tan chico: como casi ningún término aparece en muchos documentos a la vez, la penalización que aplica TF-IDF a las palabras comunes casi no cambia el resultado.

5. Consulta: “planeta tierra agua recursos naturales”

Los documentos de naturaleza volvieron a quedar arriba: `naturaleza_01` con 0,43 y `naturaleza_03` con 0,14, con el resto del corpus en cero. El score del primero es relativamente alto porque concentra varias de las palabras de la consulta (“planeta”, “tierra”), mientras que las demás coincidencias se reparten en otros textos. Esto muestra una limitación del modelo: cuando la consulta mezcla términos que aparecen en documentos distintos, el buscador no puede distinguir si se busca todo junto o cada cosa por separado.

En general, los resultados muestran que el buscador funciona bien cuando la consulta usa exactamente el vocabulario que aparece en los documentos: en las cinco consultas, el tema correcto quedó siempre en primer lugar y las categorías ajenas se fueron a cero. Los scores no son altos (rara vez pasan de 0,5) porque las consultas son cortas y comparten pocas palabras con textos que hablan de muchas cosas, pero el orden relativo es el esperado. La recuperación se vuelve menos confiable apenas la consulta usa una forma distinta de una palabra (singular en vez de plural, o un sinónimo), lo que confirma que el modelo compara palabras y no significados.

4.6. Efecto del tamaño del vocabulario

El tamaño del vocabulario determina directamente la dimensión de todos los vectores del sistema. Con nuestro corpus de 16 textos, el vocabulario resultó en 250 palabras únicas, lo que produjo vectores de 250 dimensiones.

Este número puede parecer pequeño, pero ya muestra el fenómeno de la sparsidad con claridad: el 92.80 % de los valores de la matriz son ceros. A medida que el corpus crece, esta proporción aumenta aún más, porque cada documento nuevo agrega palabras que no aparecen en los documentos anteriores, expandiendo el vocabulario sin necesariamente aumentar la densidad de cada vector.

Un vocabulario más grande tiene consecuencias en dos direcciones. Por un lado, captura más información y permite distinguir mejor entre documentos con vocabulario especializado. Por otro lado, hace que los vectores sean más dispersos, lo que puede afectar

el comportamiento del coseno en casos donde dos documentos apenas comparten términos. En esos casos, el score tiende a quedar cercano a cero incluso cuando los textos son temáticamente parecidos, simplemente porque usan sinónimos distintos.

4.7. Ventajas y limitaciones del coseno

La similitud coseno es una medida robusta para comparar documentos porque ignora la longitud de los textos y evalúa únicamente la distribución relativa de sus términos. Esto la hace efectiva para encontrar documentos con vocabulario similar sin importar si uno es más largo que el otro.

Sin embargo, tiene limitaciones importantes que vale la pena entender. La principal es que el modelo no comprende el significado de las palabras, solo detecta si aparecen o no. Dos palabras con el mismo significado pero escritas diferente (“auto” y “vehículo”, por ejemplo) son tratadas como dimensiones completamente distintas e independientes en el espacio vectorial. Como resultado, un documento sobre automóviles no necesariamente quedará cerca de uno sobre vehículos si no comparten exactamente las mismas palabras.

Otra limitación es la sensibilidad al vocabulario de la consulta. Si la consulta usa términos que no aparecen en ningún documento del corpus, su vector será el vector nulo y la similitud con cualquier documento será cero. El buscador simplemente no encontrará nada relevante.

La estrategia TF-IDF busca mejorar este panorama al reducir el peso de las palabras muy frecuentes (que aparecen en casi todos los documentos y aportan poca información) y aumentar el peso de los términos más específicos. En teoría, esto produce rankings más precisos cuando el corpus contiene documentos temáticamente diversos. En la práctica, al comparar TF y TF-IDF sobre nuestro corpus, los rankings resultaron casi iguales. Esto se explica por el tamaño reducido del conjunto: como casi ningún término aparece repartido en muchos documentos a la vez, el factor IDF apenas castiga a unas pocas palabras y el resultado se mantiene parecido al de TF. Es razonable esperar que la diferencia se note más en un corpus grande, donde sí existen palabras muy comunes a lo largo de todos los textos.

4.8. Visualización de los resultados

En el mapa de calor se nota con bastante claridad la formación de cuatro bloques a lo largo de la diagonal. Cada bloque agrupa a los documentos de un mismo tema, y dentro de él los valores de similitud son más altos (colores más intensos), mientras que fuera de él los valores caen casi a cero. Esto coincide con lo que esperábamos desde la teoría: los textos de historia comparten palabras entre sí, igual que los de matemáticas, naturaleza y tecnología, y en cambio casi no comparten vocabulario con los de otras categorías. El hecho de que estos bloques aparezcan solos, sin que nadie le dijera al programa a qué tema pertenece cada texto, muestra que la representación vectorial logra capturar la cercanía temática solo a partir de las palabras. También se ven algunos puntos tibios fuera de la diagonal, como entre matemáticas y tecnología, que se explican porque ambos temas comparten términos como “algoritmo”, “datos” o “vectores”.

En el gráfico de frecuencias se observa que las palabras que más aparecen son términos generales del corpus como “vectores”, “datos”, “matrices” o “mundo”, y no palabras vacías como artículos o preposiciones. Esto confirma que el preprocesamiento hizo bien su trabajo: al eliminar las stopwords, las palabras que quedan arriba en el conteo son las

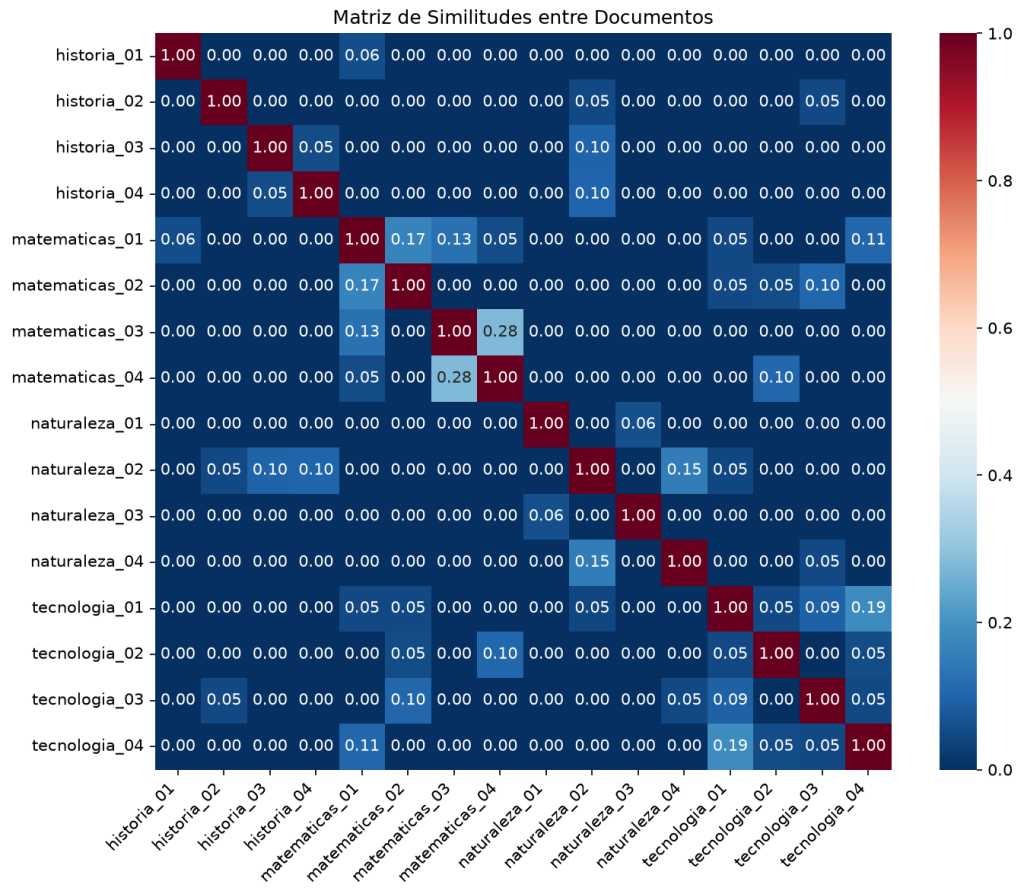


Figura 1: Mapa de calor de la similitud coseno entre los 16 documentos del corpus (vectorización TF).

que de verdad cargan significado temático. Esa es justamente la información que después permite distinguir un texto de otro.

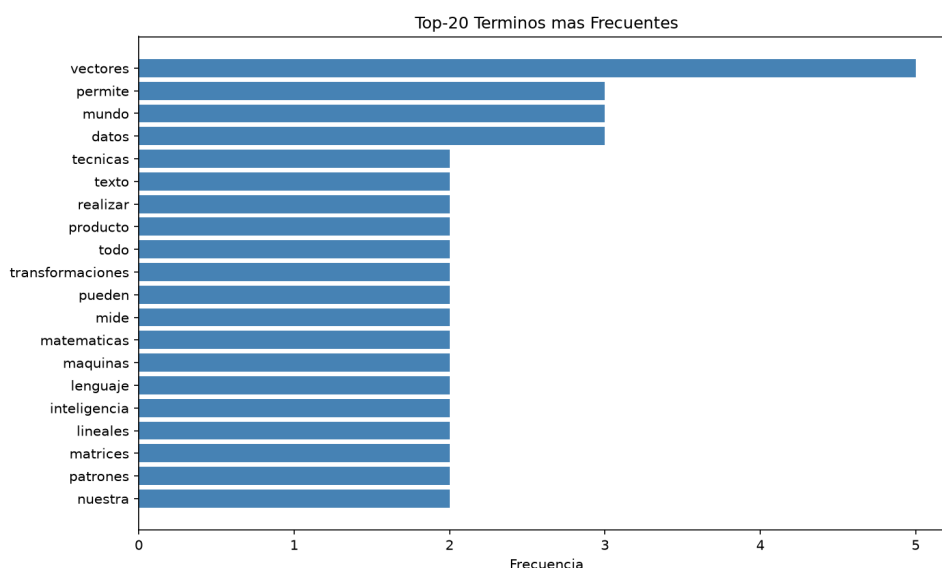


Figura 2: Términos más frecuentes en el corpus tras el preprocesamiento.

5. Reflexión y conclusiones

Construir el buscador a partir de la frecuencia de términos permitió comprobar que una idea geométrica sencilla alcanza para recuperar información con bastante precisión. Cuando la consulta usa palabras que efectivamente aparecen en los documentos, el modelo ordena los resultados de forma coherente y ubica primero los textos del tema correcto. La gran ventaja de este enfoque es su simpleza: no necesita entrenamiento previo ni grandes cantidades de datos, se apoya solo en contar palabras y medir ángulos, y aun así produce rankings razonables. Además, al trabajar con la similitud coseno, el largo de los documentos deja de ser un problema, porque lo que se compara es la dirección de los vectores y no su tamaño.

Sin embargo, el mismo análisis dejó ver con claridad dónde están los límites de un buscador que se basa únicamente en la frecuencia de términos. El problema de fondo es que el modelo no entiende el significado de las palabras, solo registra si aparecen o no. Por eso trata dos sinónimos como “auto” y “vehículo” como dimensiones distintas e independientes, y no es capaz de relacionar dos textos que hablan de lo mismo con palabras diferentes. A esto se suma su sensibilidad al vocabulario de la consulta: si alguien busca con términos que no están en ningún documento, el vector de la consulta queda en cero y la similitud con todo el corpus también, de modo que el buscador no devuelve nada. Esto se vio en la consulta que mezclaba términos repartidos en varios textos, donde los scores quedaron bajos porque las palabras no se concentraban en un solo documento. La estrategia TF-IDF ayuda en parte, porque le baja el peso a las palabras muy comunes y se lo sube a las más específicas, pero sigue sin resolver el problema del significado.

Estas mismas ideas, llevadas a una escala mucho mayor, son las que sostienen muchas de las aplicaciones que usamos todos los días. Los motores de búsqueda representan páginas y consultas como vectores para devolver los resultados más relevantes, que es exactamente lo que hace nuestro programa pero con miles de millones de documentos. Los sistemas de recomendación comparan el perfil de un usuario con el de productos o contenidos usando la misma noción de cercanía vectorial. Los chatbots y los modelos de lenguaje van un paso más allá: en lugar de contar palabras, aprenden vectores (los llama-

dos *embeddings*) que sí logran acercar palabras con significados parecidos, lo que resuelve justamente la limitación de los sinónimos que encontramos. Ver esa conexión fue quizás lo más valioso del trabajo: el buscador que construimos es una versión mínima y entendible de una herramienta que está en el corazón de la inteligencia artificial moderna, y permitió comprender desde adentro por qué el álgebra lineal es tan central en el procesamiento de texto.

6. Anexos

6.1. Lenguaje y bibliotecas

- **Lenguaje:** Python 3.10 o superior.
- **Bibliotecas:** NumPy (operaciones vectoriales y matriciales), Matplotlib y Seaborn (visualizaciones), pytest (pruebas unitarias) y Jupyter (notebook de análisis). scikit-learn se usó solo para validar el TF-IDF en las pruebas, no en el código principal.

6.2. Código

El código fuente está organizado en el repositorio del proyecto: los módulos del buscador en `src/core/` (preprocesamiento, vectorización, similitud y búsqueda), las pruebas en `tests/` y el corpus de 16 textos en `data/corpus/`.

6.3. Bibliografía

- Apuntes y material del curso de Álgebra Lineal para la Computación, Universidad Católica de Temuco.
- Manning, C. D., Raghavan, P. y Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press. (Modelo de espacio vectorial, similitud coseno y TF-IDF.)
- Documentación oficial de NumPy. <https://numpy.org/doc/>
- Documentación oficial de scikit-learn, TfidfVectorizer. https://scikit-learn.org/stable/modules/feature_extraction.html
- Documentación de Matplotlib y Seaborn para la generación de mapas de calor y gráficos de barras. <https://matplotlib.org/> y <https://seaborn.pydata.org/>